Sophonics: Reversible Local Expansion for Quantized Language Models

White Paper

`@DesperateTomatoCultivator`
`fderivaz@gmail.com`

Abstract

We present *Sophonics*, a simple architectural pattern: keep a low-bit base model frozen and attach tiny learned low-rank expansions only at high-leverage late-layer matrices. The method began inside OpenAI's Parameter Golf challenge as a quantization-repair technique and grew into a broader framework for repair, specialization, deployment, and continual adaptation. Our initial static formulation, which approximated the `int5`→`int8` residual with low-rank SVD, failed on the 17.1M competition model, recovering only 1–3% of lost performance. Switching to behavioral repair on a frozen `int5` base changed the result sharply: rank-16 learned repairs on the deepest MLP blocks recover 90.2% of the `int5`→`int8` gap on a reproduced validation slice, and the merged `int8`+zlib artifact stays under the 16 MB cap. The same late-layer repair pattern scales to open-weight Qwen models, where Qwen3-4B slightly exceeds its `int8` baseline and Qwen3-8B shows that widening the repaired late-layer window matters more than increasing rank. In the continual setting, Sophonic test-time training on Qwen2.5-1.5B and PG-19 reduces document-level deterioration by 43.4% at 16K tokens, yields 29× less cross-document contamination than full TTT when state is carried between documents, and allows saved 7.6 MB states to recover 62.5% of fresh same-document adaptation gain on a nearby continuation window. Together these results position Sophonics not as "cheaper full TTT," but as a reversible, portable local-expansion framework for low-bit language models.

1 Introduction

This work began inside OpenAI's Parameter Golf challenge [6], where quantization is already the dominant design variable rather than an afterthought. The current leaderboard is built almost entirely on int6 quantization-aware training, GPTQ-style clip search, and compression with `zstd-22`. Every bit per weight saved can be reinvested in model capacity, and every bit of quantization damage is felt directly in the final score.

We asked a natural question: could precision itself be allocated non-uniformly and dynamically, like a routable resource? Store the model at `int5` instead of the standard int6, compress the `int5`→`int8` residual into a low-rank correction, and route that correction to the layers where quantization hurts most. The idea was clean, the byte budget arithmetic worked, and the answer was no. Static precision residuals recovered almost nothing, regardless of routing strategy. The loss-relevant directions of quantization damage did not align with the directions that dominate weight-space error.

The working idea came from a different objective. Instead of preserving a higher-fidelity tensor, we learned a behavioral repair. Freeze the low-bit base, attach tiny LoRA-style [4] factors to selected weight matrices, and train only those factors against the language-model cross-entropy. The empirical finding was sharp: on the competition model, a fresh 1xRTX5090 reproduction shows

that rank-16 repairs on the deepest two MLP blocks recover 90.2% of the `int5`→`int8` gap on the 1024-sequence validation slice, while the same parameter budget on the shallowest blocks recovers only 56.5% in the localization control. The submitted competition artifact is not yet a live separated `int5`-plus-delta runtime: the repair modules are trained on a frozen `int5` base, merged back into ordinary weights, then uniformly quantized to `int8` and compressed with zlib. That artifact scores 1.353521 BPB on full validation and uses 10,861,275 bytes in total.

That competition result raised the obvious question: does this generalize? The rest of the paper follows the research outward. Scaling experiments on Qwen3-4B and Qwen3-8B show that late-layer repair transfers, with Qwen3-4B slightly exceeding its `int8` baseline and Qwen3-8B revealing a new effect where widening the repair window matters more than deepening it. Mixture experiments show that Sophonic delta packs can encode domain-specific behavior and benefit from routing. A cross-model distillation attempt produces a clean negative result: at the rank budgets tested, the repair channel is fully saturated by self-repair and cannot absorb teacher signal.

We make four claims. First, tensor fidelity is the wrong objective for quantization repair; behavioral repair via learned modules is far more effective. Second, where those modules are placed matters more than how many parameters they contain, and the high-leverage region is consistently the late feed-forward stack. Third, the same frozen-base plus late-delta pattern supports not only repair but also scaling and specialization. Fourth, in the continual setting that same pattern yields a qualitatively new object: a reversible, portable context memory with much lower cross-document interference than full-model test-time training.

2 Background and Related Work

Sophonics uses the LoRA parameterization [4]—replacing W by $W + BA$ with $r \ll \min(d_{\text{in}}, d_{\text{out}})$—but for a different purpose. LoRA adapts a full-precision model to a new task. Sophonics repairs the behavior of a frozen quantized model on its own task. QLoRA [2] bridges both ideas by quantizing for training efficiency then adapting to a new task, but it does not study *where* to place the adapter, which is central here.

The strongest independent validation comes from Schulman's *LoRA Without Regret* [8], which reports that MLP-only LoRA matches or outperforms attention-only LoRA and that adding attention on top of MLP brings little additional benefit. This matches our depth ablation exactly. Schulman also shows that LoRA matches full fine-tuning only when the dataset fits within adapter capacity—a framing that cleanly explains our distillation failure at small rank.

Our sensitivity motivation follows the spirit of GPTQ [3]: the loss impact of perturbing W_ℓ depends on the downstream Jacobian, not only on perturbation magnitude. A large Frobenius residual can be behaviorally irrelevant if its dominant directions are orthogonal to $\partial\mathcal{L}/\partial W_\ell$.

The broader framing is non-uniform resource allocation under a fixed budget. The Parameter Golf competition itself illustrates this: its live commentary [7] documents that depth recurrence amplifies quantization error $\sim$900$\times$ and that XSA helps only in the last 3–4 layers, pointing to strong depth asymmetry. Cheng et al.'s Engram [1] allocates between neural computation and scalable lookup memory; Shazeer et al. [9] route tokens across expert sub-networks. Sophonics operates on a third axis: allocating learned expansion capacity across matrices of a shared low-bit model, with the "experts" in our mixture experiments being delta packs rather than independent feed-forward branches.

The continual-learning experiments in this paper also connect to test-time training, but from a more structural angle than the usual "update the whole model" framing. Full-model TTT entangles adaptation, persistent drift, and portability because the only state being changed is the base model itself. Sophonic TTT constrains the update to explicit late-layer deltas on a frozen base. This makes

the adapted state resettable, serializable, comparable, and swappable as an object in its own right.

3 Method: Sophonic Local Expansion

Let W_ℓ denote a weight matrix in layer ℓ, and let $Q_b(W_\ell)$ denote a row-wise b-bit quantization of that matrix. The original static Sophonics formulation tried to approximate a higher-precision reference tensor by storing a low-rank residual:

$$W_{\text{eff}}^{(\ell)} = Q_b(W_\ell) + \Delta_{\text{static}}^{(\ell)}, \qquad \Delta_{\text{static}}^{(\ell)} \approx Q_8(W_\ell) - Q_b(W_\ell).$$

In practice we parameterized $\Delta_{\text{static}}^{(\ell)}$ by a truncated SVD, optionally routed to only a few layers.

The working formulation discards the fidelity objective and instead learns a small behavioral repair module:

$$W_{\text{eff}}^{(\ell)} = Q_b(W_\ell) + B_\ell A_\ell,$$

where $A_\ell \in \mathbb{R}^{r \times d_{\text{in}}}$, $B_\ell \in \mathbb{R}^{d_{\text{out}} \times r}$, and only A_ℓ, B_ℓ are trained. The base model remains frozen after quantization.

This change can be motivated by a first-order view of loss sensitivity. If a quantized model differs from its higher-precision reference by perturbations ΔW_ℓ, then for a loss $\mathcal{L}$,

$$\Delta\mathcal{L} \approx \sum_\ell \left\langle \frac{\partial \mathcal{L}}{\partial W_\ell}, \Delta W_\ell \right\rangle.$$

The directions that dominate $\|\Delta W_\ell\|_F$ or its leading singular vectors need not align with the directions that dominate $\partial\mathcal{L}/\partial W_\ell$. A low-rank SVD of the quantization residual is therefore optimizing the wrong proxy. Learned repair instead optimizes the actual task loss and lets training discover both the corrective subspace and the useful sign structure within that subspace.

We evaluate repair by how much of a low-bit gap is recovered:

$$\text{Recovery} = 100 \times \frac{m_{\text{base}} - m_{\text{repair}}}{m_{\text{base}} - m_{\text{ref}}},$$

where m is a loss-like metric with lower being better: BPB for the competition model and bits/token for the Qwen experiments.

This yields a compact operational definition.

Definition. A *sophonic delta* (*sophon*) is a small learned low-rank expansion attached to a high-leverage matrix of a frozen model, trained to restore performance or enhance behaviour.

4 Experiment 1: Static Residuals (Negative Result)

The original static-residual experiments were run on the 17.1M parameter competition model used for the Parameter Golf submission path: 9 transformer blocks, hidden size 512, grouped-query attention, and a uniform low-bit post-training quantization step. The target question was whether we could store the model at `int5`, approximate the `int5`→`int8` residual with a small rank-4 or rank-8 SVD, and route the residual budget to the layers where it helped most.

We tested three routing policies: static deepest-k, residual-norm-based selection, and random selection. All three were effectively indistinguishable. Static residual correction recovered only 1–3% of the lost performance, regardless of routing policy. Rank-4 SVD captured only 3.8% of the residual energy. The largest singular directions of the quantization residual were therefore not the directions that mattered most for language-model loss.

This negative result is not incidental; it is the reason the later formulation is credible. The failed objective was "preserve a higher-fidelity tensor." The successful objective became "restore the behavior that matters for the task." The difference turned out to be decisive.

Statistic	Result
Base / reference precisions	`int5` → `int8`
Rank-4 mean residual energy captured	3.8%
Static deepest-k routing	1–3% recovery
Residual-norm routing	1–3% recovery
Random routing	1–3% recovery

Table 1: Static precision-residual Sophonics on the 17.1M competition model.

Target set	Repaired BPB	Recovery
`blocks.0-1.mlp`	1.4092	56.5%
`blocks.7-8.mlp`	1.3610	90.2%

Table 2: Competition-model learned repair localization at `int5`. The shallow row is the matched-budget control retained in the submission notes; the `blocks.7-8.mlp` row is the fresh 2026-04-19 1024-sequence reproduction.

5 Experiment 2: Learned Repair on the Competition Model

After the static failure, we froze an `int5` base model and trained only LoRA-style repair factors on selected matrices using the ordinary cross-entropy objective. The central ablation varied *where* the repair budget was placed while keeping the parameter count and training budget matched.

The canonical submission numbers in this section come from the 2026-04-19 reproduction on a single NVIDIA GeForce RTX 5090 with 32 GB of memory. The base run used a 600 s wall-clock budget with `world_size=1`, `GRAD_ACCUM_STEPS=2`, `TRAIN_BATCH_TOKENS=131072`, and `TRAIN_SEQ_LEN=1024`, stopping at step 2975/20000. The base model's final `int8`+zlib roundtrip was val_loss = 2.25913988, val_bpb = 1.33798989.

The placement result is sharp. Repair capacity becomes dramatically more valuable toward the end of the network: the reproduced deepest-MLP repair recovers 90.2% of the `int5`→`int8` gap, while the shallow control recovers 56.5%. Broader exploratory sweeps over middle MLP blocks, late attention, MLP+attention, and wider late windows showed the same late-depth ridge, visible in Figure 1, but those exploratory rows are not the submitted artifact.

The exploratory module-type ablation further refines the picture. Late attention repair is useful, but it does not match late MLP repair, and adding attention on top of the deepest MLP blocks barely changes the outcome. This agrees well with the independent findings of Schulman [8], who reports that MLP-only LoRA can match or outperform attention-only LoRA and that attention often provides little benefit beyond MLP coverage.

The submitted competition artifact should be interpreted narrowly. During repair, the frozen `int5` base is evaluated with explicit Sophonic deltas:

$$y = Q_5(W)x + B(Ax).$$

For the final Parameter Golf package, however, those deltas are merged back into ordinary weights before export. The submitted artifact is then uniformly quantized to `int8` and compressed with zlib; it is not a runtime package of a live `int5` base plus separate Sophonic modules. Such a live separated runtime remains future work unless evaluated separately.

The repaired artifact fits comfortably under the 16 MB cap, with 5,138,725 bytes of headroom. The result proves that localized learned repair can restore much of the behavior lost to aggressive

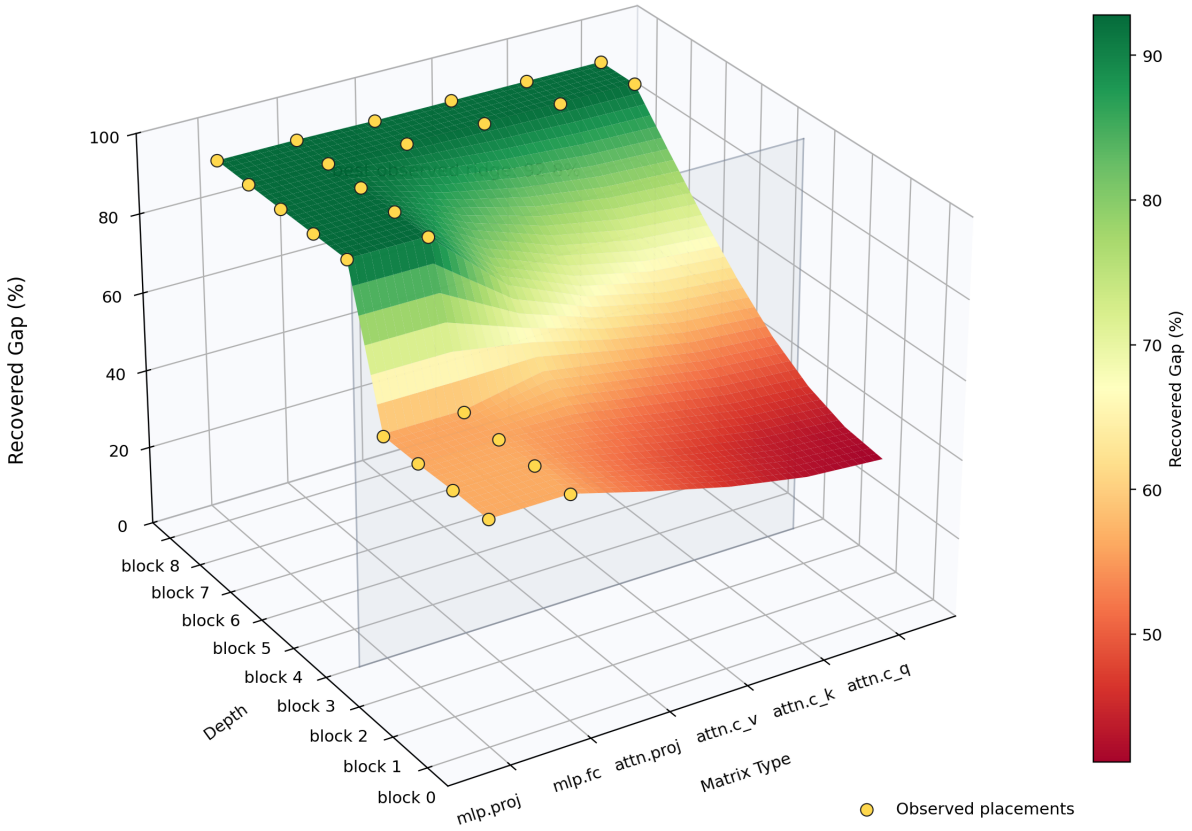

Figure 1: Exploratory Sophonic placement landscape on the competition model. The measured points form a sharp late-depth ridge, with late MLP matrices providing the highest observed recovery.

quantization and still survive a conventional `int8`+zlib export path. It does not by itself prove that the final competition package can keep the `int5` substrate and Sophonic deltas separated at inference time.

6 Experiment 3: Scaling to Qwen3-4B and Qwen3-8B

The next question was whether late-layer learned repair was only a small-model curiosity. We therefore moved to open-weight Qwen models in a Hugging Face scaffold with uniform linear `int4` proxy quantization. These experiments report bits/token rather than BPB, but the same recovery definition applies.

Two findings matter. First, the method clearly transfers. Qwen3-4B not only recovers the `int4`→`int8` gap but slightly exceeds the plain `int8` baseline. Second, the scaling bottleneck changes. At 8B, increasing rank on a fixed late-layer window helps very little:

$$\texttt{last4, r8} : 84.9\%, \qquad \texttt{last4, r12} : 85.2\%.$$

Widening the repaired late-layer window helps much more:

$$\texttt{last2, r8} : 81.5\%, \qquad \texttt{last4, r8} : 84.9\%, \qquad \texttt{last6, r8} : 86.9\%.$$

Measurement	Value
Repair target	`blocks.7-8.mlp.fc/proj`
LoRA rank / trainable params	16 / 98,304
1024-seq `int8` baseline	1.3461 BPB
1024-seq `int5` base	1.4984 BPB
1024-seq merged repair	1.3610 BPB
Final slice recovery	90.2%
256-seq eval best	1.3398 BPB, step 600, 90.6% recovery
Full val, merged fp32	2.2831 loss / 1.352204 BPB
Full val, `int8`+zlib roundtrip	2.2854 loss / 1.353521 BPB
Compressed model bytes	10,769,165
Code bytes	92,110
Total artifact bytes	10,861,275
Headroom under 16,000,000 bytes	5,138,725

Table 3: Fresh canonical Parameter Golf submission numbers for the `blocks.7-8.mlp` Sophonics repair artifact.

Model / target	`int8`	`int4`	Repaired	Recovery
Qwen3-4B, last4 `mlp+attn` (r10)	4.5247	5.3628	4.5052	102.3%
Qwen3-8B, last2 `mlp+attn` (r8)	4.2465	4.9257	4.3722	81.5%
Qwen3-8B, last4 `mlp+attn` (r8)	4.2465	4.9257	4.3487	84.9%
Qwen3-8B, last4 `mlp+attn` (r12)	4.2465	4.9257	4.3470	85.2%
Qwen3-8B, last6 `mlp+attn` (r8)	4.2465	4.9257	4.3354	86.9%

Table 4: Scaling experiments on Qwen. Recovery is measured relative to the linear `int4`→`int8` gap on the same evaluation slice.

Our interpretation is that quantization damage becomes more spatially distributed across the late stack as model size grows. The lesson from the competition model was "repair the deepest MLPs." The broader scaling law is "repair a wider late region before increasing rank." Late depth remains special, but the useful footprint broadens.

7 Experiment 4: Mixture of Sophonic Experts

If Sophonic modules only restored generic lost capacity, then domain-specialized training should matter little. We tested this on a shared Qwen3-4B `int4` base by training two packed Sophonic artifacts: a generic expert on C4 and a code specialist on CodeParrot. We then compared each expert alone, a full-strength same-window additive stack, a full-strength depth-separated stack, and an oracle router that picks the better expert per domain. For the depth-separated condition, the generic expert lived on layers 32–33 and the code expert on layers 34–35; both same-window experts used the true last 4 layers.

The specialization result is clear. The generic expert is best on C4, the code specialist is best on code, and the oracle router improves the equal-weight mean from 3.1097 or 3.1134 down to 3.0457. This is only possible if the Sophonic modules encode domain-specific behavior rather than acting as interchangeable generic slack.

The compositional result is also clear, but negative. Full-strength additive stacking harms both domains. The same-window stack rises to 4.5107 on C4 and 1.9050 on code, while the depth-separated stack reaches 4.5200 and 1.8862 respectively. In this recipe, simple depth separation does

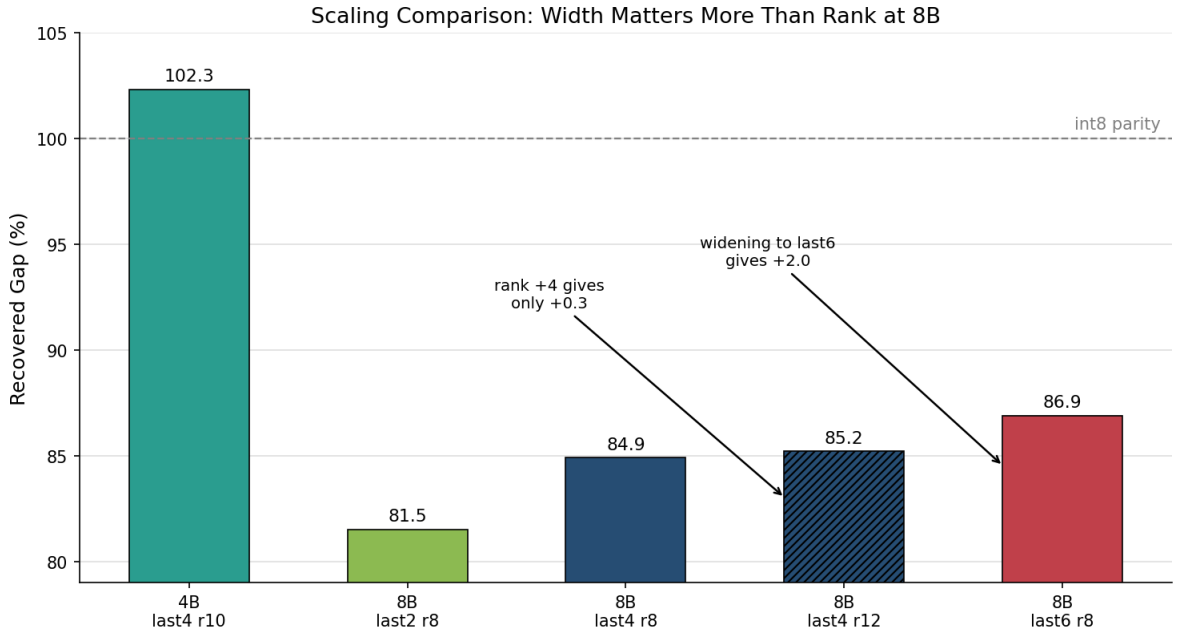

Figure 2: Scaling comparison on Qwen. The 4B model slightly exceeds its `int8` baseline, while the 8B model shows a clear width > rank effect: widening the repaired late-layer window helps more than increasing rank on a fixed window.

Policy	C4	Code	Mean
Generic same-window	4.3227	1.8967	3.1097
Code same-window	4.4581	1.7687	3.1134
Same-window stack (1.0 + 1.0)	4.5107	1.9050	3.2079
Depth-separated stack (1.0 + 1.0)	4.5200	1.8862	3.2031
Oracle per-domain routing	4.3227	1.7687	3.0457

Table 5: Mixture of Sophonic Experts (MoSE) on Qwen3-4B `int4`. Lower bits/token is better. The rightmost column is the equal-weight mean of C4 and code evaluations. The strongest result is routed specialization, not full-strength simultaneous composition.

not rescue simultaneous composition.

To test whether the failure came from forcing both experts to act at full amplitude, we also evaluated down-weighted auxiliary blends. On C4, `generic + 0.1 * code` reaches 4.3240 bits/token versus 4.3227 for the generic expert alone. On code, `code + 0.1 * generic` reaches 1.7692 versus 1.7687 for the code expert alone. At larger weights, performance degrades steadily. This is not a positive stacking result, but it is also not a total collapse: low-weight auxiliary blending is nearly neutral. In this recipe, shrinkage helps more than depth separation.

The resulting picture is sharper than the earlier draft. Sophonic modules can be trained as a modular library on one frozen substrate, and they can be selected or routed successfully by domain. What we do not yet have is a strong beneficial simultaneous composition rule. Routing works cleanly; full-strength addition does not; low-weight blending is stable but not additive.

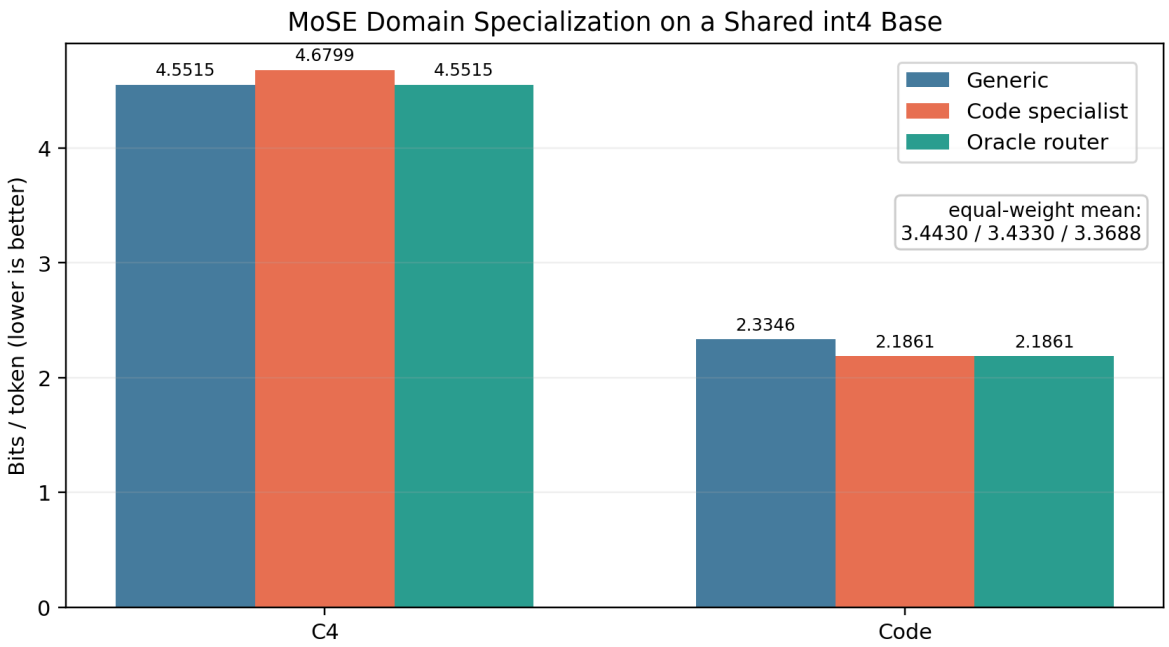

Figure 3: Domain specialization in MoSE. The generic expert is best on C4, the code expert is best on code, and routed selection beats both same-window and depth-separated full-strength composition.

Objective	Recovery	Outcome
Pure distillation ($\lambda = 0.5$)	90.9%	below self-repair
Hybrid CE+KL ($\lambda = 0.1$)	97.2%	still below 4.5052
Code-specific distillation	–	worse than code specialist on code and C4

Table 6: Cross-model distillation into a Qwen3-4B `int4` student with late-layer deltas.

8 Experiment 5: Cross-Model Distillation (Negative Result)

The next natural question was whether the same low-rank repair channel could carry not only self-repair but also some of the behavior of a larger teacher. We therefore trained a Qwen3-4B `int4` student with late-layer Sophonic deltas against a frozen Qwen3-8B teacher using

$$\mathcal{L} = \mathcal{L}_{\mathrm{CE}} + \lambda T^2 \, \mathrm{KL}\Big(p_T^{\mathrm{teacher}} \parallel p_T^{\mathrm{student}}\Big) .$$

The result was negative.

Pure distillation trailed ordinary self-repair. A lighter hybrid loss improved substantially, but it still failed to beat the plain self-repair baseline of 4.5052 bits/token. Code-specific distillation was worse than the plain code specialist on both code *and* general text.

The simplest explanation is capacity saturation. At rank 8–10, the late-layer delta channel is already fully occupied repairing the quantization damage of the student itself. Once that repair subspace is saturated, teacher supervision is not free extra signal; it is noise competing for the same limited corrective degrees of freedom. This interpretation is consistent with the capacity framing in [8].

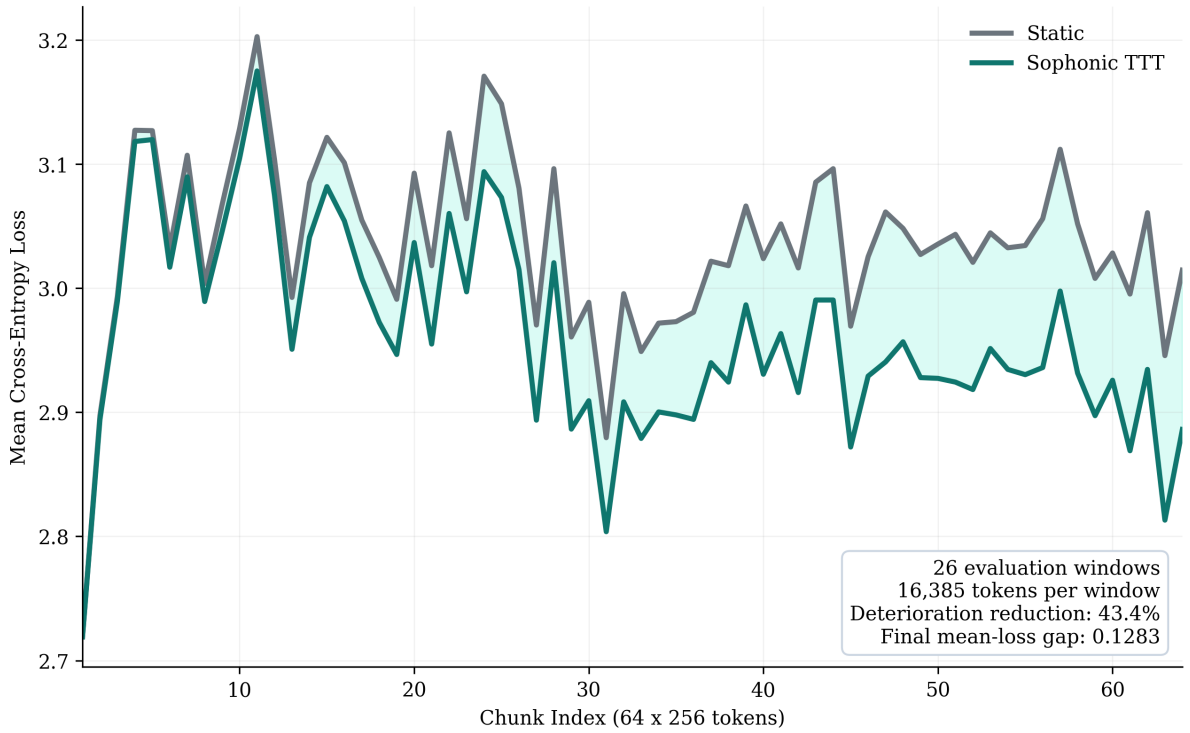

Figure 4: Long-horizon continual adaptation on Qwen2.5-1.5B and PG-19. Averaged over 26 A40 evaluation windows of length 16,385 tokens, Sophonic TTT remains below static throughout the document and reduces overall deterioration by 43.4%.

9 Experiment 6: Continual Test-Time Adaptation

The same frozen-base plus late-delta pattern can also be used online. Instead of training Sophonic modules once and keeping them fixed, we attach late-layer LoRA-style factors to a frozen base model and update only those factors as a document unfolds. This turns the adapter into an explicit state object. It can be reset, carried across documents, saved, reloaded, and compared as an artifact in its own right. This is not merely a cheaper version of full-model test-time training. It has properties that full TTT fundamentally cannot have, because the base weights remain untouched.

We evaluated continual adaptation on Qwen2.5-1.5B with PG-19 windows using a chunked left-to-right protocol. Each document window is split into fixed-size chunks, loss is measured on every chunk, and the adaptive method receives one gradient step after each chunk except the last. The default Sophonic continual configuration targets the last 6 layers, covers both MLP and attention matrices, and uses rank-8 factors. GPU references were run on an A40 in `bfloat16`; the same positive trend also appeared locally on an M2 MacBook Air, where full-model TTT was not feasible.

9.1 Long-Horizon Continual Adaptation

The large-scale continual result is straightforward. On 26 PG-19 evaluation windows of length 16,385 tokens (64 chunks of 256 tokens), static evaluation deteriorates from 2.7191 to 3.0144 mean loss ($\Delta = +0.2954$). Sophonic TTT deteriorates from 2.7191 to 2.8861 ($\Delta = +0.1671$). That is a 43.4% reduction in document-level deterioration relative to static evaluation. The effect also strengthens with horizon: the corresponding matched local result at the same 64×256 window length was 23.6%, and shorter 16×256 and 32×256 pilots were weaker still.

Method	Params	Final gain	Mean gain	sec/chunk	adapt sec/chunk
Sophonic TTT	1.98M (0.128%)	+0.1055	+0.0476	0.16	0.13
Full TTT	1.55B (100%)	+0.2266	+0.1178	0.28	0.26

Table 7: Matched continual comparison on Qwen2.5-1.5B, PG-19, and 64×256 token windows. "Gain vs static" measures the reduction in final or mean loss relative to the static baseline on the same 4 documents.

The most important point is not a single endpoint number but the direction of the trend. The late-layer delta interface becomes more useful, not less, as the adaptation horizon grows. This matters because the strongest objection to small continual adapters is that they might capture only the earliest part of a document and then plateau. In these runs the signal does not plateau within 16K tokens.

9.2 Matched Full-TTT Baseline

The strongest baseline for the continual setting is full-model TTT on the same model, document set, and chunking schedule. We therefore ran a matched A40 comparison on 4 documents at 64×256, using a stable full-TTT learning rate of 3×10^{-5}.

This result forced a cleaner efficiency framing than the one we began with. Sophonic does *not* support an "orders of magnitude cheaper compute" claim in wall-clock terms. On this matched setup it recovers 46.6% of the full-TTT final-chunk gain at 57.1% of total chunk time and 49.8% of the adaptation overhead. The right interpretation is narrower but still useful: Sophonic is competitive in adaptation-per-extra-update cost, while keeping the mutable state tiny. At this setting the trainable state is 1.98M parameters rather than 1.55B, corresponding to roughly 15 MB of Adam state rather than roughly 12 GB. That difference is operationally large: Sophonic ran locally on an M2 laptop, while full TTT produced `nan` in `float16` and OOMed in `float32`.

9.3 Carryover Safety and Swappable Context Memory

The most novel continual results are not the matched timing numbers. They are the safety and portability properties that come from freezing the base model.

First, we tested what happens when the adapted state is deliberately carried from one document into the next rather than being reset. Full TTT carryover starts the next document 0.1133 loss worse than the static baseline on the first chunk. Sophonic carryover starts only 0.0039 worse. That is a 29× reduction in cross-document contamination. This is not "zero forgetting," but it is a large and directly measurable safety advantage in any sequential deployment where the model processes unrelated documents back to back.

Second, we treated the adapted Sophonic state as an artifact. A state trained on PG-19 document 1, window 0 was serialized to a 7.6 MB checkpoint, reloaded later, and evaluated without any further optimization. On the nearby continuation, document 1, window 1, the loaded state reduced the final loss from 3.4219 to 3.3438. Fresh live adaptation on that same window reduced it to 3.2969. So the loaded state recovers 62.5% of the benefit of fresh adaptation on a nearby continuation window. It also does so at roughly inference-only cost: about 0.03 s/chunk rather than 0.16 s/chunk for live adaptation. But the same saved state hurts unrelated documents: on document 0, window 0 the final loss rises from 2.5469 to 2.6094, and on document 2, window 0 it rises from 3.4531 to 3.4688. The saved state is therefore not a generic improvement. It is a document-specific context memory.

This is the strongest qualitative result of the continual section. The same architectural choice that makes Sophonic reversible also makes it portable. The adapted state can be reset instantly, saved to disk, reloaded later, or shared between users working on the same document or codebase.

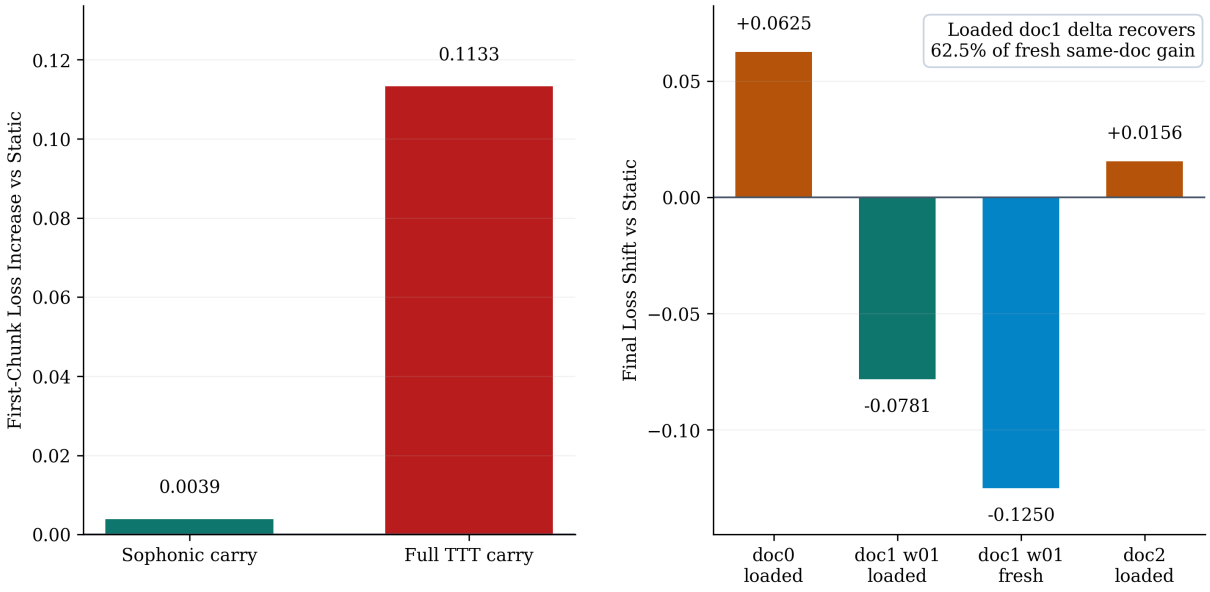

Figure 5: Safety and portability of continual Sophonic states. Left: when state is carried across documents, Sophonic exhibits far less first-chunk contamination than full TTT. Right: a saved state from PG-19 document 1, window 0 helps on the nearby continuation, recovering 62.5% of fresh same-document adaptation gain, but hurts unrelated documents.

Full TTT cannot expose its adaptation in this form because the adaptation is baked directly into the base weights.

10 Discussion

The research arc supports a simple conceptual shift. The first Sophonics idea was about *precision as a routable resource*. That failed because high-fidelity reconstruction of a higher-precision tensor is not the same as preserving behavior. The working Sophonics idea is about *behavioral repair at high-leverage intervention points*. This perspective makes sense of every major result in the paper.

With the continual results included, Sophonics should no longer be read as a single quantization-repair trick. The same frozen-base plus late-delta pattern now supports three distinct modes of use: offline repair of an aggressively quantized model, routed domain specialization, and online context adaptation. That is why the strongest framing is architectural rather than purely benchmark-driven.

The depth ablation is the anchor result. On the competition model, it shows a clear late-MLP ridge that cannot be explained by "LoRA anywhere helps." The Qwen experiments generalize the same lesson with a nuance: late depth is still where the repair budget belongs, but at 8B the effective footprint becomes wider and includes attention more meaningfully. In other words, the high-leverage region broadens with scale.

The MoSE experiments show that Sophonic deltas are not merely generic compression patches. They can encode domain-specific behavior, and the best current composition story is routed selection rather than full-strength simultaneous addition. The strongest same-window generic and code experts are each excellent on their home domains; oracle routing beats both same-window and depth-separated full-strength stacks; and a 0.1-weight auxiliary blend is nearly neutral but still not better than the best single expert. In this recipe, shrinking the secondary expert helps more than separating it by depth. This places Sophonics near MoE and Engram conceptually, but on a different axis. MoE routes tokens across subnetworks [9]; Engram routes between neural computation and

scalable lookup [1]; Sophonics allocates a fixed budget of extra capacity across matrices of a shared low-bit model.

The negative distillation result is equally informative. It suggests that the repair adapter is a narrow channel whose first obligation is to undo low-bit damage. If that channel is saturated, extra teacher supervision will not create free capability transfer. This is one of the clearest cases where a negative result sharpens the method rather than weakening it.

The continual section changes the interpretation of efficiency. The wall-clock story is modest: Sophonic is cheaper than full TTT, but not dramatically so. The stronger advantages are memory, reversibility, and portability. Because the base model is frozen, Sophonic adaptation can be reset instantly, saved as a context-specific artifact, and loaded again later. The cross-document carryover experiment shows why this matters: full TTT entangles adaptation with persistent drift, while Sophonic keeps the mutable state small and contained. The transfer experiment then strengthens the point further by showing that the saved state is useful on a nearby continuation yet harmful on unrelated documents, exactly as one would expect from a document-specific memory.

11 Limitations and Future Work

Several limitations remain.

- The Qwen experiments still use simplified training-time quantization surrogates rather than AWQ, GPTQ, or other production-grade grouped quantization stacks. A fully matched training recipe against such formats remains open.
- The strongest placement evidence is concentrated on late layers. We did not exhaustively explore conditional routing, token-dependent activation of Sophonic modules, or broader architectural variants.
- The MoSE result uses oracle domain routing. A learned router over prompts, domains, or even token windows remains open.
- The current composition tests cover only generic and code-specialist Sophonic modules. We did not yet test whether a continual-adaptation Sophonic state composes with repair or specialist modules, whether low-weight blending can be learned rather than hand-set, or whether token-level gating changes the outcome.
- The distillation failure was observed at small rank. Larger or hierarchical repair channels might admit some teacher transfer after self-repair is already handled.
- The continual experiments currently center on one open-weight base model (Qwen2.5-1.5B) and one long-form benchmark (PG-19). Broader document types, code repositories, and conversational logs remain to be tested.
- The continual save/load results currently use raw PyTorch checkpoints. More compact serialization, retrieval, and indexing for large banks of context memories remain open systems questions.
- The compute claim is limited. Sophonic does not yet show a dramatic wall-clock frontier over full TTT; its strongest advantages are lower mutable-state footprint and qualitative properties of the frozen-base interface.

These limitations suggest a clearer agenda. The highest-value next steps are conditional or routed Sophonics, training recipes that close the proxy-quantizer gap against production-grade quantization stacks, larger-scale transfer matrices over saved context states, and more compact serialization for libraries of portable deltas. The central lesson is not tied to one quantizer: small learned local expansions can buy back a large fraction of the behavior lost to aggressive compression, and in the continual setting they can also act as explicit context objects rather than hidden changes

to the model.

12 Conclusion

Sophonics started as a precision-routing idea and became a learned local-expansion method. The static residual version mostly failed. The learned-repair version worked on the competition model, scaled to 4B and 8B open-weight models, produced domain-specialized delta packs whose strongest composition story is routed selection rather than full-strength additive stacking, resisted overclaiming in the face of a negative distillation result, and finally extended to continual adaptation where the same late-layer delta interface acts as a reversible context memory.

The unified definition that emerges is straightforward: a sophonic delta (sophon) is a small learned low-rank expansion attached to a high-leverage matrix of a frozen model, trained to restore performance or enhance behaviour. The base insight is not that low-bit models can be made perfect. It is that a cheap low-bit substrate plus a very small amount of carefully placed learned expansion can often recover most of what aggressive quantization takes away, and because the base remains frozen, the learned state becomes reversible, portable, and in the continual case directly reusable as a document-specific memory.

The compositional lesson is narrower but important. Sophonic modules already behave like a modular library in the routed sense: a generic expert can be selected for generic text, a code expert can be selected for code, and that oracle policy beats any full-strength simultaneous composition we tested. What the current results do *not* show is beneficial unweighted stacking. Same-window and depth-separated full-strength addition both degrade performance, while very small auxiliary blends are nearly neutral but still not positive. The next step is therefore not to assume that "more Sophonics added together" works automatically, but to treat routing, gating, and learned composition weights as first-class parts of the method.

References

[1] Xin Cheng, Weizhe Zeng, Damai Dai, Qingquan Chen, Benyou Wang, Zhifang Xie, and others. Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models. *arXiv preprint arXiv:2601.07372*, 2026.

[2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.

[3] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, 2022.

[5] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*, 2020.

[6] OpenAI. Parameter Golf. GitHub repository, 2026. https://github.com/openai/parameter-golf.

[7] OpenAI. Parameter Golf Live AI Commentary, Issue #140. GitHub issue, 2026. https://github.com/openai/parameter-golf/issues/140.

[8] John Schulman and Thinking Machines Lab. LoRA Without Regret. *Thinking Machines Lab: Connectionism*, 2025. https://thinkingmachines.ai/blog/lora/.

[9] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations*, 2017.